

SQL JOIN & SUBQUERY Practice Questions

BASIC LEVEL QUESTIONS

1. Retrieve employee names with department names using INNER JOIN.
2. Display all customers with their orders.
3. Find students along with enrolled course names.
4. Show patients with doctor names using JOIN.
5. Retrieve employees who belong to IT department.
6. Display products with sales quantity.
7. Find orders placed by customers from Mumbai.
8. Retrieve employees whose salary is greater than average salary.
9. Find students who scored above average marks.
10. Show doctors whose bill amount is greater than average bill.

INTERMEDIATE LEVEL QUESTIONS

11. Find employees who earn the highest salary in each department.
12. Display customers who placed more than 2 orders.
13. Retrieve products never sold using LEFT JOIN.
14. Find students enrolled in more than one course.
15. Show departments having average salary greater than 70000.
16. Retrieve patients who visited more than once.
17. Find second highest salary using subquery.
18. Display courses with total revenue generated.
19. Find doctors generating highest revenue.
20. Retrieve employees hired after the employee named Rahul.

HARD LEVEL QUESTIONS

21. Find top 3 highest paid employees using subquery.
22. Retrieve customers whose total order amount is above overall average revenue.
23. Find employees who earn more than their department average salary.
24. Display department-wise second highest salary.
25. Retrieve products contributing more than 20% of total sales.
26. Find students scoring highest marks in each course.
27. Show doctors treating maximum number of patients.
28. Retrieve customers who ordered in consecutive months.
29. Find employees whose salary is equal to maximum salary in another department.
30. Display cumulative sales revenue using window function.

SQL JOIN & SUBQUERY Solutions

1. Employee names with department names

```
SELECT e.employee_name, d.department_name
FROM Employees e
JOIN Departments d
ON e.department_id = d.department_id;
```

2. Customers with their orders

```
SELECT c.customer_name, o.order_id
FROM Customers c
JOIN Orders o
ON c.customer_id = o.customer_id;
```

3. Students with enrolled course names

```
SELECT s.student_name, c.course_name
FROM Students s
JOIN Enrollments e
ON s.student_id = e.student_id
JOIN Courses c
ON e.course_id = c.course_id;
```

4. Patients with doctor names

```
SELECT p.patient_name, d.doctor_name
FROM Patients p
JOIN Appointments a
ON p.patient_id = a.patient_id
JOIN Doctors d
ON a.doctor_id = d.doctor_id;
```

5. Employees in IT department

```
SELECT e.employee_name
FROM Employees e
JOIN Departments d
ON e.department_id = d.department_id
WHERE d.department_name = 'IT';
```

6. Products with sales quantity

```
SELECT p.product_name, s.quantity  
FROM Products p  
JOIN Sales s  
ON p.product_id = s.product_id;
```

7. Orders placed by customers from Mumbai

```
SELECT o.order_id, c.customer_name  
FROM Orders o  
JOIN Customers c  
ON o.customer_id = c.customer_id  
WHERE c.city = 'Mumbai';
```

8. Employees with salary above average

```
SELECT *  
FROM Employees  
WHERE salary > (  
SELECT AVG(salary)  
FROM Employees  
);
```

9. Students scoring above average marks

```
SELECT *  
FROM Enrollments  
WHERE marks > (  
SELECT AVG(marks)  
FROM Enrollments  
);
```

10. Doctors with bill amount above average

```
SELECT *  
FROM Appointments  
WHERE bill_amount > (  
SELECT AVG(bill_amount)  
FROM Appointments  
);
```

11. Highest salary in each department

```
SELECT *  
FROM Employees e  
WHERE salary = (  
SELECT MAX(salary)  
FROM Employees  
WHERE department_id = e.department_id  
);
```

12. Customers with more than 2 orders

```
SELECT customer_id, COUNT(*) AS total_orders  
FROM Orders  
GROUP BY customer_id  
HAVING COUNT(*) > 2;
```

13. Products never sold

```
SELECT p.*  
FROM Products p  
LEFT JOIN Sales s  
ON p.product_id = s.product_id  
WHERE s.product_id IS NULL;
```

14. Students enrolled in more than one course

```
SELECT student_id, COUNT(*) AS total_courses  
FROM Enrollments  
GROUP BY student_id  
HAVING COUNT(*) > 1;
```

15. Departments with average salary > 70000

```
SELECT department_id, AVG(salary) AS avg_salary  
FROM Employees  
GROUP BY department_id  
HAVING AVG(salary) > 70000;
```

16. Patients who visited more than once

```
SELECT patient_id, COUNT(*) AS visits
FROM Appointments
GROUP BY patient_id
HAVING COUNT(*) > 1;
```

17. Second highest salary

```
SELECT MAX(salary) AS second_highest
FROM Employees
WHERE salary < (
SELECT MAX(salary)
FROM Employees
);
```

18. Courses with total revenue generated

```
SELECT c.course_name,
COUNT(e.student_id) * c.fees AS revenue
FROM Courses c
JOIN Enrollments e
ON c.course_id = e.course_id
GROUP BY c.course_name, c.fees;
```

19. Doctors generating highest revenue

```
SELECT d.doctor_name,
SUM(a.bill_amount) AS revenue
FROM Doctors d
JOIN Appointments a
ON d.doctor_id = a.doctor_id
GROUP BY d.doctor_name
ORDER BY revenue DESC
LIMIT 1;
```

20. Employees hired after Rahul

```
SELECT *  
FROM Employees  
WHERE hire_date > (  
  SELECT hire_date  
  FROM Employees  
  WHERE employee_name = 'Rahul'  
);
```

21. Top 3 highest paid employees

```
SELECT *  
FROM Employees  
ORDER BY salary DESC  
LIMIT 3;
```

22. Customers with total order amount above average revenue

```
SELECT customer_id, SUM(total_amount) AS total_revenue  
FROM Orders  
GROUP BY customer_id  
HAVING SUM(total_amount) > (  
  SELECT AVG(total_amount)  
  FROM Orders  
);
```

23. Employees earning more than department average

```
SELECT *  
FROM Employees e  
WHERE salary > (  
  SELECT AVG(salary)  
  FROM Employees  
  WHERE department_id = e.department_id  
);
```

24. Department-wise second highest salary

```
SELECT department_id,  
MAX(salary) AS second_highest  
FROM Employees e1  
WHERE salary < (  
SELECT MAX(salary)  
FROM Employees e2  
WHERE e1.department_id = e2.department_id  
)  
GROUP BY department_id;
```

25. Products contributing more than 20% of total sales

```
SELECT product_id,  
SUM(quantity) AS total_quantity  
FROM Sales  
GROUP BY product_id  
HAVING SUM(quantity) > (  
SELECT SUM(quantity) * 0.2  
FROM Sales  
);
```

26. Students scoring highest marks in each course

```
SELECT *  
FROM Enrollments e1  
WHERE marks = (  
SELECT MAX(marks)  
FROM Enrollments e2  
WHERE e1.course_id = e2.course_id  
);
```

27. Doctors treating maximum patients

```
SELECT doctor_id,  
COUNT(patient_id) AS total_patients  
FROM Appointments  
GROUP BY doctor_id  
ORDER BY total_patients DESC  
LIMIT 1;
```


28. Customers ordering in consecutive months

```
SELECT customer_id,  
MONTH(order_date) AS order_month  
FROM Orders;
```

29. Employees with salary equal to another department maximum

```
SELECT *  
FROM Employees  
WHERE salary IN (  
SELECT MAX(salary)  
FROM Employees  
GROUP BY department_id  
);
```

30. Cumulative sales revenue

```
SELECT order_date,  
SUM(total_amount) OVER (  
ORDER BY order_date  
) AS cumulative_revenue  
FROM Orders;
```